



K-VLSI

UVM PROGRAMS

NAME: DHANAPAL K

KVLSI ID: KVLSI2501061

Under the guidance of :

Shashi Kant Sharma

INDEX

Sl No.	CONTENTS	Page No.
1.	Verification of 4x1 MUX in UVM	01 – 07
2.	Object Creation and Randomization in UVM	08 – 10
3.	Phases of UVM	11 – 16



1. Verification of 4x1 MUX in UVM

- **Code**

```
module mux41 (intf inf);

    always_comb
    begin
        case(inf.S)
            2'b00: inf.Y=inf.I[0];
            2'b01: inf.Y=inf.I[1];
            2'b10: inf.Y=inf.I[2];
            2'b11: inf.Y=inf.I[3];
            default inf.Y=1'bx;
        endcase
    end

endmodule

// //-----DUT-----//
interface intf;

    logic [3:0] I;
    logic [1:0] S;
    logic Y;

endinterface

// ////////////////////////////////////TB////////////////////////////////////

//`include "uvm_pkg.sv"
`include "uvm_macros.svh"
import uvm_pkg::*;

////////////////////////////////////
class transaction extends uvm_sequence_item;
`uvm_object_utils(transaction)

    rand bit [3:0]I;
    randc bit [1:0]S;
    bit Y;
```

```
//constraint c1 { $onehot(I);}
function new(input string name = "transaction");
    super.new(name);
endfunction
```

```
endclass
```

```
////////////////////////////////////
```

```
class generator extends uvm_sequence#(transaction);
`uvm_object_utils(generator)
```

```
    rand transaction tr;
```

```
function new(input string name = "generator");
    super.new(name);
endfunction
```

```
virtual task body();
repeat(15)
    begin
        tr = transaction::type_id::create("tr");
        start_item(tr);
        assert(tr.randomize());
        `uvm_info("SEQ", $sformatf("I : %b S : %b Y : %b", tr.I, tr.S, tr.Y), UVM_NONE);
        finish_item(tr);
    end
endtask
```

```
endclass
```

```
////////////////////////////////////
```

```
class seqr extends uvm_sequencer#(transaction);
`uvm_component_utils(seqr)
```

```
function new(input string inst = "seqr", uvm_component parent = null);
    super.new(inst,parent);
endfunction
```

```
endclass
```

```
////////////////////////////////////
```

```

class drv extends uvm_driver#(transaction);
  `uvm_component_utils(drv)

  transaction tr;
  virtual intf vif;

  function new(input string name = "drv", uvm_component parent = null);
    super.new(name,parent);
  endfunction

  virtual function void build_phase(uvm_phase phase);
  super.build_phase(phase);
  if(!uvm_config_db#(virtual intf)::get(this,"", "vif",vif))//uvm_test_top.env.agent.drv.aif
    `uvm_fatal("drv","Unable to access Interface");
    tr = transaction::type_id::create("tr");
  endfunction

  virtual task run_phase(uvm_phase phase);
    super.run_phase(phase);

    forever begin
      seq_item_port.get_next_item(tr); //gen2drv.get(tr);
      vif.I <= tr.I; // same
      vif.S <= tr.S; //same
      `uvm_info("DRV", $sformatf("I : %b S : %b Y : %b", tr.I, tr.S, tr.Y), UVM_NONE); // same
      seq_item_port.item_done();
      #20; //same
    end
  endtask

endclass

////////////////////////////////////
class mon extends uvm_monitor;
  `uvm_component_utils(mon)

  uvm_analysis_port#(transaction) send;
  transaction tr;
  virtual intf vif;

  function new(input string inst = "mon", uvm_component parent = null);
    super.new(inst,parent);
  endfunction

  virtual function void build_phase(uvm_phase phase);
  super.build_phase(phase);
  tr = transaction::type_id::create("tr"); //handle creation for transaction

```

```

    send = new("send", this);// create the handle of ANalysis
    if(!uvm_config_db#(virtual intf)::get(this,"","vif",vif))//uvm_test_top.env.agent.driv.aif
        `uvm_fatal("mon","Unable to access Interface");
    endfunction

    virtual task run_phase(uvm_phase phase);
    forever begin
        #20;
        tr.I = vif.I;
        tr.S = vif.S;
        tr.Y = vif.Y;
        `uvm_info("MON", $sformatf("I : %b S : %b Y : %b", tr.I, tr.S, tr.Y), UVM_NONE);
        send.write(tr);
    end
    endtask

endclass

// //////////////////////////////////////
class sco extends uvm_scoreboard;
`uvm_component_utils(sco)

    uvm_analysis_imp#(transaction,sco) recv;

    function new(input string inst = "sco", uvm_component parent = null);
        super.new(inst,parent);
    endfunction

    virtual function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        recv = new("recv", this);
    endfunction

    virtual function void write(transaction tr);
        logic expected_y;
        case(tr.S)
            2'b00: expected_y=tr.I[0];
            2'b01: expected_y=tr.I[1];
            2'b10: expected_y=tr.I[2];
            2'b11: expected_y=tr.I[3];
            default expected_y=1'bx;
        endcase

        if(expected_y==tr.Y)
            `uvm_info("SCO", $sformatf("Test Passed -> I : %b S : %b Y : %b ", tr.I, tr.S, tr.Y),
            UVM_NONE)

```

```
        else
            `uvm_error("SCO", $sformatf("Test Passed -> I : %b S : %b Y : %b ", tr.I, tr.S, tr.Y))

        $display("-----");
    endfunction

endclass

// //////////////////////////////////////

class agent extends uvm_agent;
    `uvm_component_utils(agent)

    function new(input string inst = "agent", uvm_component parent = null);
        super.new(inst,parent);
    endfunction

    drv d;
    seqr sqr;
    //uvm_sequencer#(transaction) seqr;
    mon m;

    virtual function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        d = drv::type_id::create("d",this);
        m = mon::type_id::create("m",this);
        sqr = seqr::type_id::create("sqr", this);
    endfunction

    virtual function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        d.seq_item_port.connect(sqr.seq_item_export);
    endfunction

endclass

// //////////////////////////////////////

class env extends uvm_env;
    `uvm_component_utils(env)

    function new(input string inst = "env", uvm_component c);
        super.new(inst,c);
    endfunction

    agent a;
    sco s;
```

```

virtual function void build_phase(uvm_phase phase);
super.build_phase(phase);
  a = agent::type_id::create("a",this);
  s = sco::type_id::create("s", this);
endfunction

virtual function void connect_phase(uvm_phase phase);
super.connect_phase(phase);
a.m.send.connect(s.recv); // connecting thw anlaysis piort of Monitor with Analsis export of
Scoreboard
endfunction

endclass

////////////////////////////////////
class test1 extends uvm_test;
`uvm_component_utils(test1)

function new(input string inst = "test", uvm_component c);
super.new(inst,c);
endfunction

env e; // handle for Environment
generator gen; // handle for Sequence

//genrator1 gen1;

virtual function void build_phase(uvm_phase phase);
super.build_phase(phase);
  e = env::type_id::create("env",this);
  gen = generator::type_id::create("gen");
endfunction

virtual function void end_of_elaboration_phase(uvm_phase phase);
super.end_of_elaboration_phase(phase);
uvm_top.print_topology();
endfunction

virtual task run_phase(uvm_phase phase);
phase.raise_objection(this);
gen.start(e.a.sqr); // start sequence over the sequencer
#20;
phase.drop_objection(this);
endtask

```



```
// ////////////////////////////////////////  
module tb;  
  
    intf inf(); // instantiate the Interface  
  
    mux41 dut (inf); // Instantiate the DUT  
  
    initial  
    begin  
        uvm_config_db #(virtual intf)::set(null, "*", "vif", inf);  
        run_test("test1");  
    end  
  
    initial begin  
        $dumpfile("dump.vcd");  
        $dumpvars;  
    end  
endmodule
```

- **Output:**

--- UVM Report Summary ---

** Report counts by severity

UVM_INFO : 64

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 0

** Report counts by id

[DRV] 15

[MON] 15

[RNTST] 1

[SCO] 15

[SEQ] 15

[TEST_DONE] 1

[UVM/RELNOTES] 1

[UVMTOP] 1

- **EDA Playground Link:**

<https://www.edaplayground.com/x/X83r>

2. Object Creation and Randomization in UVM

- **Code**

```
`include "uvm_macros.svh"
`include "uvm_pkg.sv"
import uvm_pkg::*;

// Define class before module
class my_object extends uvm_object;

    rand bit [7:0] a;
    rand bit [3:0] b;

    `uvm_object_utils_begin(my_object)
        `uvm_field_int(a, UVM_ALL_ON)
        `uvm_field_int(b, UVM_ALL_ON)
    `uvm_object_utils_end

    function new(string name = "my_object");
        super.new(name);
    endfunction

endclass

module test;

    my_object m1, m2;

    initial
    begin
        m1 = my_object::type_id::create("Obj1");
        m2 = my_object::type_id::create("Obj2");

        repeat (2)
        begin
            assert(m1.randomize());
            m1.print();
            m2.copy(m1);
            m2.print();
        end

        `uvm_info("MSG1", "This is the first Message", UVM_NONE)
        `uvm_warning("MSG2", "This is the second Message")
        `uvm_error("MSG3", "This is the third Message")
        `uvm_fatal("MSG4", "This is the fourth Message")
        `uvm_info("MSG5", "This is the fifth Message", UVM_NONE)
```

endmodule

- **Output:**

```
-----  
Name Type      Size Value  
-----  
Obj1 my_object - @112  
  a integral 8   'hb9  
  b integral 4   'hc  
-----
```

```
-----  
Name Type      Size Value  
-----  
Obj2 my_object - @113  
  a integral 8   'hb9  
  b integral 4   'hc  
-----
```

```
-----  
Name Type      Size Value  
-----  
Obj1 my_object - @112  
  a integral 8   'h5b  
  b integral 4   'h0  
-----
```

```
-----  
Name Type      Size Value  
-----  
Obj2 my_object - @113  
  a integral 8   'h5b  
  b integral 4   'h0  
-----
```

```
UVM_INFO testbench.sv(39) @ 0: reporter [MSG1] This is the first Message  
UVM_WARNING testbench.sv(40) @ 0: reporter [MSG2] This is the second Message  
UVM_ERROR testbench.sv(41) @ 0: reporter [MSG3] This is the third Message  
UVM_FATAL testbench.sv(42) @ 0: reporter [MSG4] This is the fourth Message  
UVM_INFO /apps/vcsmx/vcs/U-2023.03-SP2/etc/uvm-ieee/src/base/uvm_report_server.svh(902)  
@ 0: reporter [UVM/REPORT/SERVER]  
--- UVM Report Summary ---
```

```
** Report counts by severity  
UVM_INFO : 2  
UVM_WARNING : 1  
UVM_ERROR : 1  
UVM_FATAL : 1  
** Report counts by id
```

[MSG1] 1
[MSG2] 1
[MSG3] 1
[MSG4] 1
[UVM/RELNOTES] 1

- **EDA Playground Link:**

<https://www.edaplayground.com/x/aSPq>

3. Phases of UVM

- **Code**

```
class driver extends uvm_driver;

  `uvm_component_utils(driver)

  function new(string name= "driver", uvm_component parent= null);
    super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    `uvm_info("DRV_BUILD","Hi I am in drivers build phase", 0)
  endfunction

  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    `uvm_info("DRV_connect","Hi I am in drivers connect phase", 0)
  endfunction

endclass


class monitor extends uvm_monitor;

  `uvm_component_utils(monitor)

  function new(string name= "monitor", uvm_component parent= null);
    super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    `uvm_info("MON_BUILD","Hi I am in monitors build phase", 0)
  endfunction

  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    `uvm_info("MON_connect","Hi I am in monitors connect phase", 0)
  endfunction

endclass
```

```
class sequencer extends uvm_sequencer;

  `uvm_component_utils(sequencer)

  function new(string name= "sequencer", uvm_component parent= null);
    super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    `uvm_info("SQR_BUILD","Hi I am in sequencers build phase", 0)
  endfunction

  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    `uvm_info("SQR_connect","Hi I am in sequencers connect phase", 0)
  endfunction

endclass


class agent extends uvm_agent;

  `uvm_component_utils(agent)

  driver drv;
  monitor mon;
  sequencer sqr;

  function new(string name= "agent", uvm_component parent= null);
    super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);

    drv= driver::type_id::create("DRV", this);
    mon= monitor::type_id::create("MON", this);
    sqr= sequencer::type_id::create("SQR", this);
    `uvm_info("agent_BUILD","Hi I am in agents build phase", 0)
  endfunction

  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    `uvm_info("agent_connect","Hi I am in agents connect phase", 0)
```

```
endfunction

endclass

class env extends uvm_env;

    `uvm_component_utils(env)

    agent a;

    function new(string name= "env", uvm_component parent= null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);

        a= agent::type_id::create("AGT", this);

        `uvm_info("ENV_BUILD","Hi I am in envs build phase", 0)
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        `uvm_info("ENV_connect","Hi I am in ENVs connect phase", 0)
    endfunction

endclass

class generator extends uvm_driver;

    `uvm_component_utils(driver)

    function new(string name= "driver", uvm_component parent= null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        `uvm_info("DRV_BUILD","Hi I am in drivers build phase", 0)
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
```

```
`uvm_info("DRV_connect","Hi I am in drivers connect phase", 0)
endfunction

endclass

class test extends uvm_test;

`uvm_component_utils(test)

env e;

function new(string name= "test", uvm_component parent= null);
    super.new(name,parent);
endfunction

function void build_phase(uvm_phase phase);
    super.build_phase(phase);

    e= env::type_id::create("ENV", this);

    `uvm_info("Test_BUILD","Hi I am TESTs build phase", 0)
endfunction

function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    `uvm_info("TEST_connect","Hi I am TESts connect phase", 0)
endfunction

function void end_of_elaboration_phase(uvm_phase phase);
    super.end_of_elaboration_phase(phase);
    uvm_top.print_topology();
    `uvm_info("TEST_EOB","Hi I am TESts EOB phase", 0)
endfunction

endclass

///-----

`include "uvm_macros.svh"
`include "uvm_pkg.sv"
import uvm_pkg::*;

module top;

    initial begin
        run_test("test");
```


- **Output:**

Name	Type	Size	Value
uvm_test_top	test	-	@336
ENV	env	-	@349
AGT	agent	-	@360
DRV	driver	-	@372
rsp_port	uvm_analysis_port	-	@391
seq_item_port	uvm_seq_item_pull_port	-	@381
MON	monitor	-	@401
SQR	sequencer	-	@410
rsp_export	uvm_analysis_export	-	@419
seq_item_export	uvm_seq_item_pull_imp	-	@537
arbitration_queue	array	0	-
lock_queue	array	0	-
num_last_reqs	integral	32	'd1
num_last_rsps	integral	32	'd1

UVM_INFO testbench.sv(176) @ 0: uvm_test_top [TEST_EOB] Hi I am TESts EOB phase
UVM_INFO /apps/vcsmx/vcs/U-2023.03-SP2/etc/uvm-1.2/src/base/uvm_report_server.svh(904) @
0: reporter [UVM/REPORT/SERVER]
--- UVM Report Summary ---

**** Report counts by severity**

UVM_INFO : 16
UVM_WARNING : 0
UVM_ERROR : 0
UVM_FATAL : 0

**** Report counts by id**

[DRV_BUILD] 1
[DRV_connect] 1
[ENV_BUILD] 1
[ENV_connect] 1
[MON_BUILD] 1
[MON_connect] 1
[RNTST] 1
[SQR_BUILD] 1
[SQR_connect] 1
[TEST_EOB] 1
[TEST_connect] 1
[Test_BUILD] 1
[UVM/RELNOTES] 1

[UVMTOP] 1
[agent_BUILD] 1
[agent_connect] 1

- **EDA Playground Link:**

<https://www.edaplayground.com/x/uBQu>